```
#include <vector>
#include <atomic>
#include <iostream>

std::vector<int> data;
std::atomic<bool> data_ready(false);

void reader_thread()
{
    while(!data_ready.load())    <-1
    {
        std::this_thread::sleep(std::milliseconds(1));
    }
    std::cout<<"The answer="<<data[0]<<"\n";    <-2
}
void writer_thread()
{
    data.push_back(42);        <-3
    data_ready=true;     <-4
}
```

Leaving aside the inefficiency of the loop waiting for the data to be ready ❶, you really need this to work, because otherwise sharing data between threads becomes impractical: every item of data is forced to be atomic. You've already learned that it's undefined behavior to have nonatomic reads ❷ and writes ❸ accessing the same data without an enforced ordering, so for this to work there must be an enforced ordering somewhere.

The required enforced ordering comes from the operations on the `std::atomic<bool>` variable `data_ready`; they provide the necessary ordering by virtue of the memory model relations *happens-before* and *synchronizes-with*. The write of the data ❸ happens-before the write to the `data_ready` flag ❹, and the read of the flag ❶ happens-before the read of the data ❷. When the value read from `data_ready` ❶ is `true`, the write synchronizes-with that read, creating a happens-before relationship. Because happens-before is transitive, the write to the data ❸ happens-before the write to the flag ❹, which happens-before the read of the `true` value from the flag ❶, which happens-before the read of the data ❷, and you have an enforced ordering: the write of the data happens-before the read of the data and everything is OK. Figure 5.2 shows the important happens-before relationships in the two threads. I've added a couple of iterations of the `while` loop from the reader thread.

Figure 5.2. Enforcing an ordering between nonatomic operations using atomic operations